SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1737

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration:** 1 Hour
Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ***Saai Vardhan D (192525205)*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Question 1 – Water Jug Problem [10 marks]

1. Problem understanding

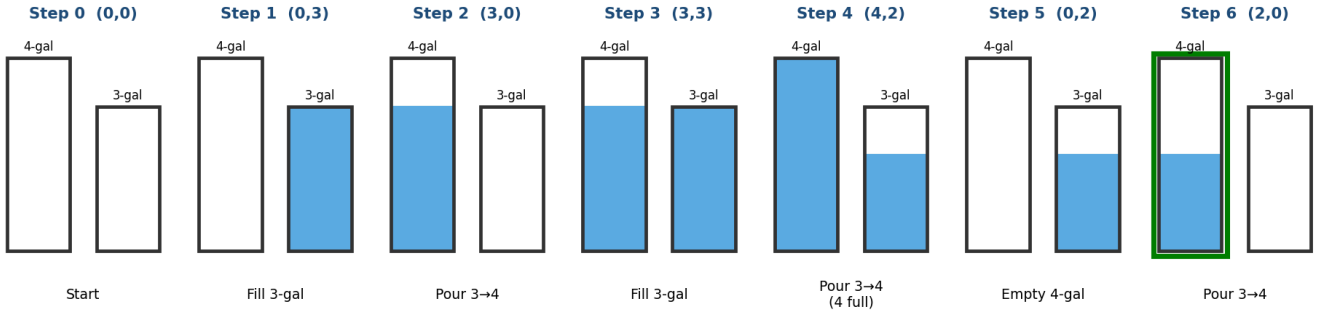
Two jugs (4-gal and 3-gal) with no markings, a pump and a drain. **Goal: exactly 2 gallons in the 4-gallon jug.** Represent a state as **(x, y)** where x = water in the 4-gal jug (0–4) and y = water in the 3-gal jug (0–3). **Initial state = (0, 0); goal state = (2, n)** for any n.

2. Method – state-space search (BFS)

Every legal operation moves the system from one state to another. Breadth-First Search explores all states level by level, so the first time it reaches (2, n) it has found the **shortest** sequence of operations.

#	Condition	New state	Operation
1	$x < 4$	(4, y)	Fill the 4-gal jug from the pump
2	$y < 3$	(x, 3)	Fill the 3-gal jug from the pump
3	$x > 0$	(0, y)	Empty the 4-gal jug on the ground
4	$y > 0$	(x, 0)	Empty the 3-gal jug on the ground
5	$x + y \geq 4, y > 0$	$(4, y - (4 - x))$	Pour 3-gal into 4-gal until the 4-gal is full
6	$x + y \geq 3, x > 0$	$(x - (3 - y), 3)$	Pour 4-gal into 3-gal until the 3-gal is full
7	$x + y \leq 4, y > 0$	(x + y, 0)	Pour all of the 3-gal into the 4-gal
8	$x + y \leq 3, x > 0$	(0, x + y)	Pour all of the 4-gal into the 3-gal

3. Solution (6 operations)

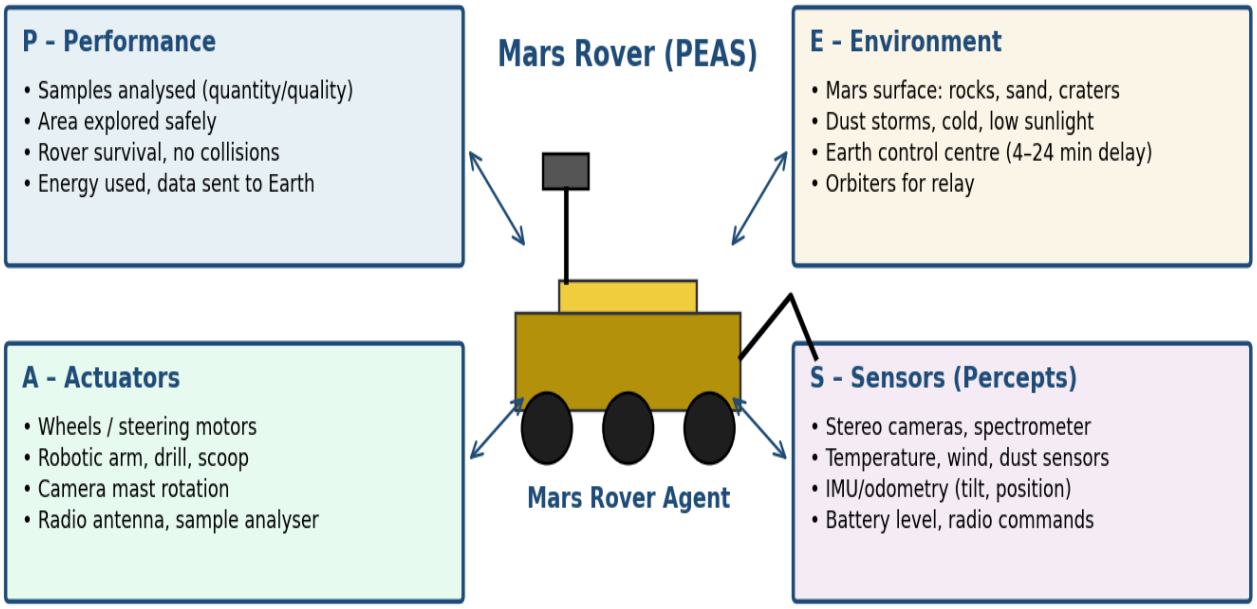


Step	Rule applied	State (x, y)	Meaning
1	Rule 2 – fill 3-gal	(0, 0) → (0, 3)	3-gal jug is full
2	Rule 7 – pour 3 → 4	(0, 3) → (3, 0)	4-gal now holds 3
3	Rule 2 – fill 3-gal	(3, 0) → (3, 3)	Both jugs hold 3
4	Rule 5 – pour 3 → 4 until full	(3, 3) → (4, 2)	Only 1 gal fits, 2 gal remain in 3-gal
5	Rule 3 – empty 4-gal	(4, 2) → (0, 2)	4-gal is empty
6	Rule 7 – pour 3 → 4	(0, 2) → (2, 0)	**Goal: 2 gallons in the 4-gal jug**

4. Interpretation

The goal (2, 0) is reached in the minimum of 6 steps. Check: 3 + 3 − 4 = 2 gal. This works because gcd(4, 3) = 1, so any whole number of gallons up to 4 can be measured. A second 6-step solution also exists: (0,0)→(4,0)→(1,3)→(1,0)→(0,1)→(4,1)→(2,3).

Question 2 – Mars Rover Agent [10 marks]



i. Percepts

Stereo/panoramic camera images; spectrometer and chemical readings of rocks and soil; temperature, wind and dust sensor values; wheel odometry and IMU data (position, tilt, slope); motor current (wheel stuck/slipping); battery and solar-panel level; radio commands from Earth.

ii. Operating environment

Property	Value	Reason
Observable	Partially	Sensors see only nearby terrain; hidden rocks, craters and sand pits are unseen
Agents	Single agent	No competing agents; Earth only sends high-level commands
Deterministic	Stochastic	Wheel slip, dust storms and sensor noise make outcomes uncertain
Episodic	Sequential	A driving decision affects later position, energy and sampling choices
Static	Dynamic	Weather, light and temperature change while the rover is deciding
Discrete	Continuous	Terrain, position, time and sensor values are continuous
Known	Largely unknown	Unexplored terrain, so the rover must learn; 4–24 min radio delay forces autonomy

iii. Actions

Move forward/backward, steer/turn, stop; rotate mast and camera; extend robotic arm; drill or scoop a sample; place sample in analyser and run spectrometer; store or discard sample; avoid obstacle / re-plan route; transmit data to Earth or orbiter; sleep and recharge.

iv. Performance measure

Performance = w1·(samples analysed × scientific value) + w2·(area explored safely) + w3·(data returned) − w4·(damage/risk) − w5·(energy used). Also judged by rover survival time (mission life) and number of collisions or stuck events.

v. Suitable architecture – hybrid goal-based + utility-based, model-based, layered learning agent

- **Model-based:** the environment is only partially observable, so it keeps an internal map, position and battery state.
- **Goal-based:** with a 4–24 minute delay it cannot be remote-controlled; it must plan by itself (reach rock X, drill, analyse).
- **Utility-based:** it must trade off science value, energy and safety, which a utility function handles.
- **Reflex layer + learning:** reflexes (stop at a cliff edge, tilt limit) give instant safety; learning improves terrain handling. A simple reflex agent alone would fail.

Question 3 – 8-Queens Problem [10 marks]

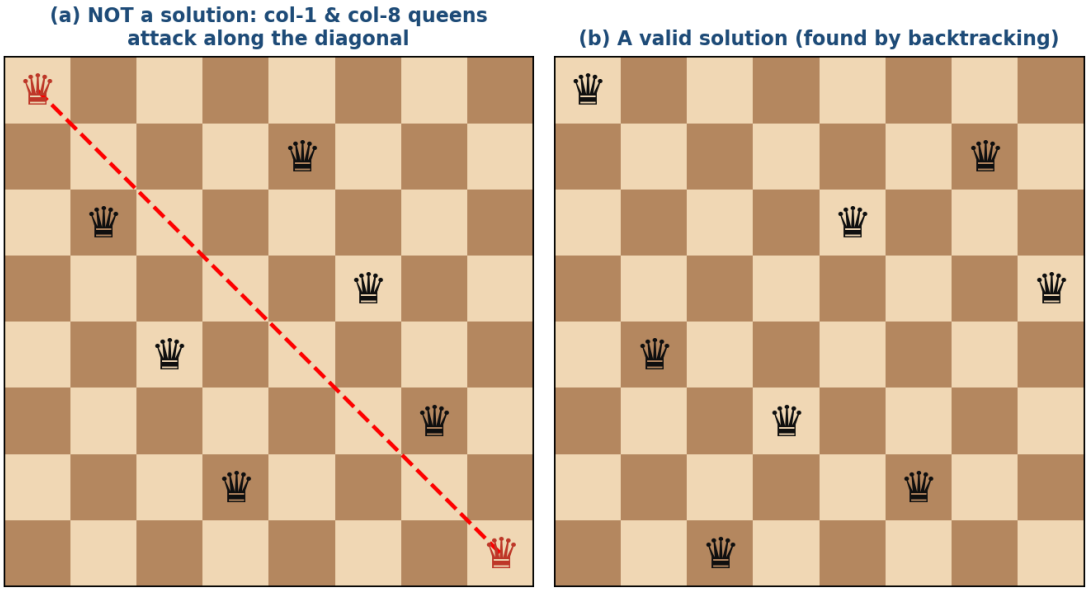


Figure (a) is not a solution: the queen in the first column and the queen in the last column are on the same diagonal (red). Figure (b) is a valid solution.

Problem formulation (incremental)

Component	Definition
Initial state	Empty board (0 queens)
States	Any arrangement of 0 to 8 queens on the board in which no queen attacks another (queens added column by column)
Actions	Add a queen to the leftmost empty column, in a row not attacked by any queen already placed
Transition model	Returns the board with the new queen added
Goal test	8 queens on the board and no two share a row, column or diagonal
Path cost	Not important (only the final arrangement matters); each step costs 1

Exploring the search space

Formulation	States	Comment
Complete-state, any 8 queens placed anywhere	$\approx 1.8 \times 10^{14}$	Hopelessly large
One queen per column (8 rows each)	$8^8 \approx 16.7$ million	Large
Incremental, no attacked placement	2,057 states	Small, practical

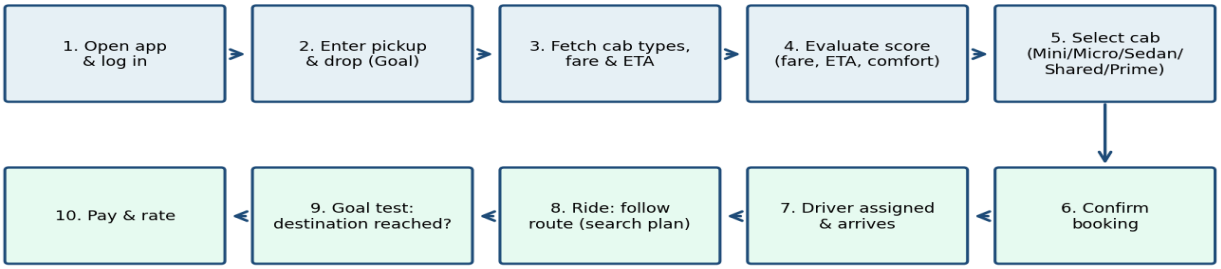
- **BFS** is complete but must hold every partial board at each level in memory (large frontier).
- **DFS with backtracking** is best: every solution is at depth exactly 8, memory is only O(8), and a dead end (a column with no safe row) simply backtracks to the previous column.
- **Result:** 92 solutions exist (12 distinct if rotations/reflections are ignored). Example solution (row of queen in columns 1–8): 1, 5, 8, 6, 3, 7, 2, 4.

Question 4 – OLA Cab Booking Agent [10 marks]

Type of agent

A **goal-based problem-solving agent** (with a utility-based element for choosing the cab type). The goal is to reach the destination; the agent formulates a problem, searches for the best (cab type, route) and then executes the plan.

Formulation	OLA example
Percepts	Pickup, drop, time, available cab types, fare, ETA, traffic
Initial state / Goal	Customer at pickup location P / Customer at destination D
Actions	Select cab (mini, micro, sedan, shared, prime…), confirm booking, ride along route
Cost / utility	Fare, ETA and comfort; the best cab maximises the score
Environment	Observable (app shows all cabs), mostly deterministic, dynamic (traffic), single customer



Pseudocode

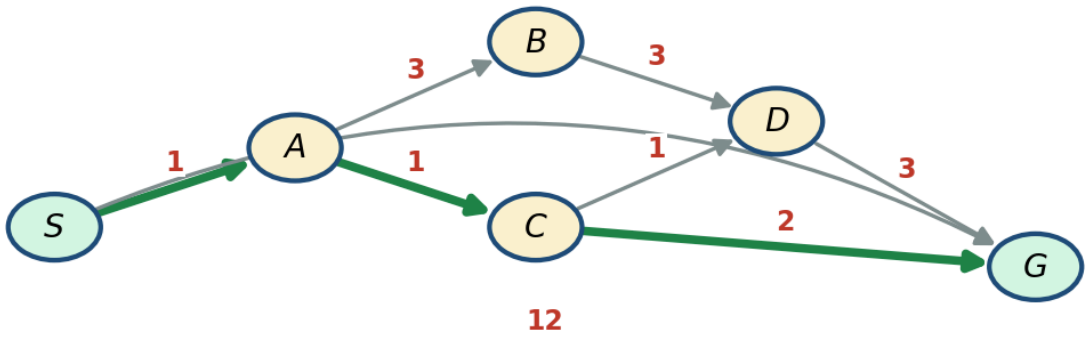
```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence (initially empty)
              state, goal, problem
  state <- UPDATE-STATE(state, percept)      // pickup P, drop D, cab list
  if seq is empty then
     goal    <- FORMULATE-GOAL(state)         // be at destination D
     problem <- FORMULATE-PROBLEM(state, goal)
     seq     <- SEARCH(problem)               // = BOOK-CAB below
     if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq);  seq <- REST(seq)
  return action

procedure BOOK-CAB(P, D, user_preference)
  cabs <- {mini, micro, sedan, shared, prime, ...}
  best <- null;  best_score <- -infinity
  for each c in cabs do
     if AVAILABLE(c, P) then
        fare <- FARE(c, P, D);  eta <- ETA(c, P)
        score <- w1*COMFORT(c) - w2*fare - w3*eta
        if score > best_score then best <- c;  best_score <- score
  if best = null then return failure          // retry / change pickup
  chosen <- (user selects cab) or best
  CONFIRM-BOOKING(chosen);  ASSIGN-DRIVER(chosen)
  repeat TRACK-RIDE() until current_location = D     // goal test
  PAY-AND-RATE();  return success
```

Question 5 – Uniform Cost Search (UCS) [10 marks]

1. Problem understanding

Find the least-cost path from **S** to **G** in a weighted directed graph. Edges: S→A 1, S→G 12, A→B 3, A→C 1, B→D 3, C→D 1, C→G 2, D→G 3.



Green path S → A → C → G = optimal (cost 4)

2. Method – why UCS

Edge costs differ, so BFS (fewest edges) would pick S→G (cost 12). UCS always expands the node with the lowest path cost g(n) from a priority queue, so the first time G is removed from the queue its path is optimal.

3. Solution process

Step	Node expanded (g)	Children generated (g)	Frontier (priority queue)
1	S (0)	A (1), G (12)	A1, G12
2	A (1)	B (4), C (2)	C2, B4, G12
3	C (2)	D (3), G (4) – cheaper than 12, so G is updated	D3, B4, G4
4	D (3)	G (6) – rejected, 6 > 4	B4, G4
5	B (4)	D (7) – rejected, D already expanded at 3	G4
6	G (4)	Goal removed from queue – stop	–

Order of expansion: S, A, C, D, B, G. **Path found:** S → A → C → G, **cost = 1 + 1 + 2 = 4**.

Other paths	Cost	Result
S → A → C → D → G	6	Costlier
S → A → B → D → G	10	Costlier
S → G (direct)	12	Costliest

4. Interpretation

The cheapest delivery route is **S → A → C → G with a total cost of 4 units (fuel + time)**. This saves 8 units (about 67%) compared with the direct route S → G. UCS is complete and optimal because every edge cost is positive.